```
  "name":"Elasticsearch crash course",
  "tags":["Elasticsearch","Software engineering", "Programming"]
}

PUT tech_books/_doc/3
{
  "name":"Java Core Fundamentals",
  "tags":["Java","Software"]
}
```

Now that we've seeded our book inventory index, users can search for books using simple `match/term` queries. (I'm omitting these queries here because we've already mastered them.) However, not all user queries yield results; for example, someone searching for *Python in Action* won't find it. The following listing shows this.

Listing 12.28 Searching for a nonexistent book

```
GET tech_books/_search
{
  "query": {
    "match": {
      "name": "Python"
    }
  }
}
```

From the user's perspective, the search ends with a disappointing result: not returning the book they searched for. We can take our queries to the next level by notifying the user when the out-of-stock Python book becomes available. This is where we can put percolators to work.

Just as we index documents into an index, a percolator can have an index for a set of queries and expect the queries to be indexed. We need to define a schema for a `percolator` index. Let's call it `tech_books_percolator`.

Listing 12.29 Creating a `percolator` index

```
PUT tech_books_percolator
{
  "mappings": {
    "properties": {
      "query": {
        "type": "percolator"
      },
      "name": {
        "type": "text"
      },
      "tags": {
        "type": "text"
      }
    }
  }
}
```

Sets the field name to query

Sets the query field's data type to percolator

Sets the name field as it appears in the tech_books index

Sets the tags field as it appears in the tech_books index